Semantics

Semantics

Syntax: concerned with the grammatical structure of programs (what sequences of characters constitute programs? Grammars, lexers, parsers, automata theory.)

e.g z:=x; x:=y; y:=z (syntactic analysis)

Semantics : meaning of grammatically correct program (what does a program mean (do)? When are two programs equivalent? When does a program satisfy its specification?)

Semantics

What does a program do?

Hard to get it right!

What does the following C statement print “x==1” ?

printf(“%d %d\n”, x++, ++x);

Can we replace “f(x)+f(x)” with “2*f(x)” ?

What does a program mean?

Compile and run

– Implementation dependencies

– Not useful for reasoning

Informal Semantics (Reference manual)

– Natural language description of PL

Formal Semantics

– Description in terms of notation with formally understood meaning

Types

Operational semantics: The meaning of a construct is specified by the computation it induces when it is executed on a machine. In particular, it is of interest how the effect of a computation is produced.

Denotational semantics: Meanings are modelled by mathematical objects that represent the effect of executing the constructs. Thus only the effect is of interest, not how it is obtained.

Axiomatic semantics: Specific properties of the effect of executing the constructs are expressed as assertions. Thus there may be aspects of the executions that are ignored.

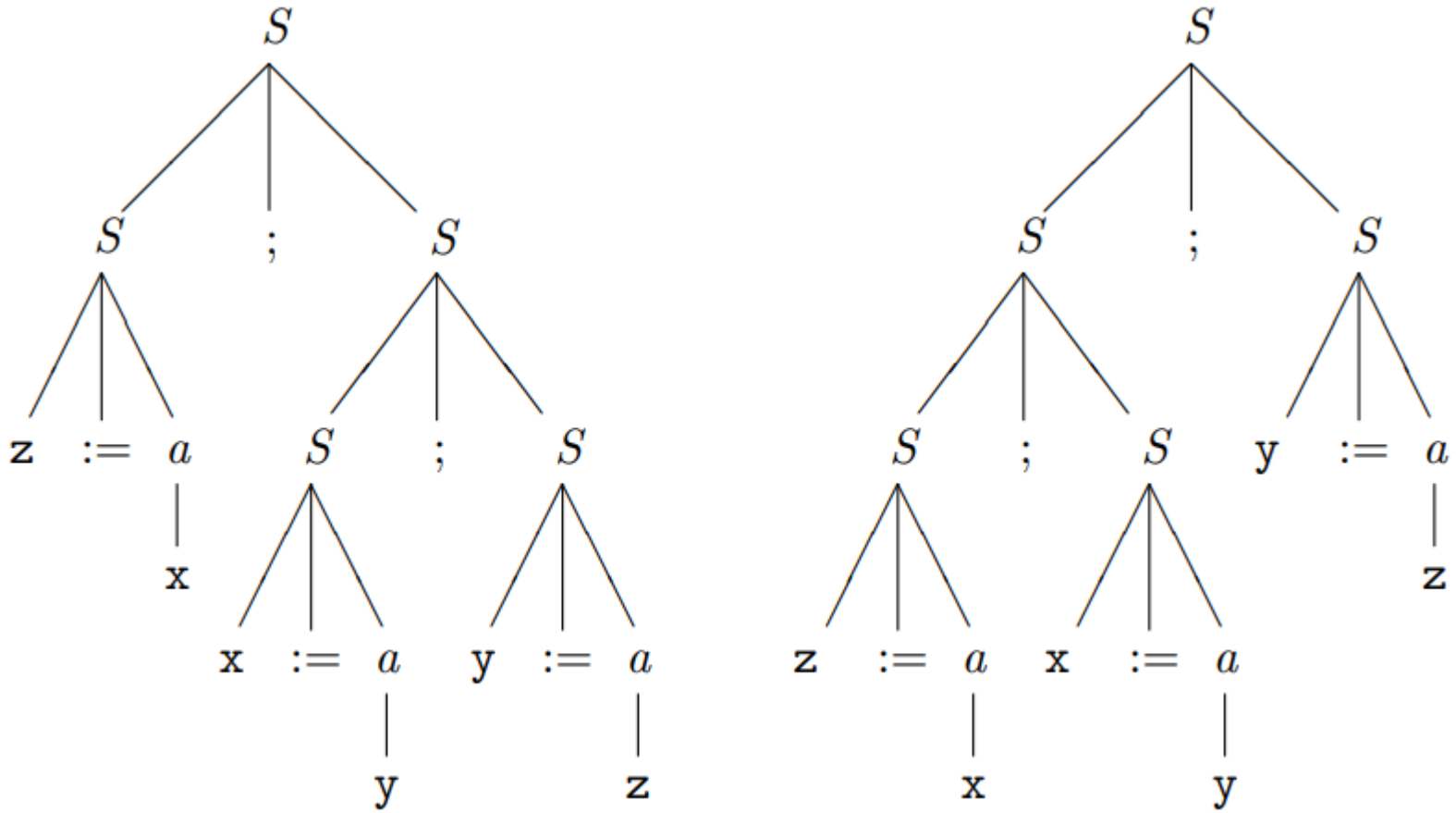
Semantics of While (Imperative Programming Language)

BNF

n will range over numerals= Num
x will range over variables= Var
a will range over arithmetic expressions= Aexp
b will range over boolean expressions= Bexp
S will range over statements= Stm.

a ::= n | x | a1 + a2 | a1 * a2 | a1 – a2
b ::= true | false | a1 = a2 | a1 ≤ a2 | ¬b | b1 ∧ b2
S ::= x := a | skip | S 1 ; S 2 | if b then S 1 else S 2
| while b do S

$$
\begin{aligned}
S \;::=\; & x := a \;\mid\; \texttt{skip} \;\mid\; S_1; S_2 \\
& \mid \texttt{if}\ b\ \texttt{then}\ S_1\ \texttt{else}\ S_2 \\
& \mid \texttt{while}\ b\ \texttt{do}\ S \\
& \mid \texttt{repeat}\ S\ \texttt{until}\ b
\end{aligned}
$$



Abstract Syntax Tree: z:=x; (x:=y; y:=z) and (z:=x; x:=y); y:=z

Semantics of Expressions

$$
\begin{aligned}
\mathcal{A}[\![n]\!]s &= \mathcal{N}[\![n]\!] \\
\mathcal{A}[\![x]\!]s &= s\ x \\
\mathcal{A}[\![a_1 + a_2]\!]s &= \mathcal{A}[\![a_1]\!]s + \mathcal{A}[\![a_2]\!]s \\
\mathcal{A}[\![a_1 \star a_2]\!]s &= \mathcal{A}[\![a_1]\!]s \cdot \mathcal{A}[\![a_2]\!]s \\
\mathcal{A}[\![a_1 - a_2]\!]s &= \mathcal{A}[\![a_1]\!]s - \mathcal{A}[\![a_2]\!]s
\end{aligned}
$$

Example:

$$s\ \mathbf{x} = \mathbf{3}$$

$$
\begin{aligned}
\mathcal{A}[\![\mathbf{x}+1]\!]s &= \mathcal{A}[\![\mathbf{x}]\!]s + \mathcal{A}[\![1]\!]s \\
&= (s\ \mathbf{x}) + \mathcal{N}[\![1]\!] \\
&= \mathbf{3} + \mathbf{1} \\
&= \mathbf{4}
\end{aligned}
$$

Semantics of boolean expression

$$\mathcal{B}[\![\texttt{true}]\!]s = \mathbf{tt}$$

$$\mathcal{B}[\![\texttt{false}]\!]s = \mathbf{ff}$$

$$\mathcal{B}[\![a_1 = a_2]\!]s = \begin{cases} \mathbf{tt} & \text{if } \mathcal{A}[\![a_1]\!]s = \mathcal{A}[\![a_2]\!]s \\ \mathbf{ff} & \text{if } \mathcal{A}[\![a_1]\!]s \neq \mathcal{A}[\![a_2]\!]s \end{cases}$$

$$\mathcal{B}[\![a_1 \leq a_2]\!]s = \begin{cases} \mathbf{tt} & \text{if } \mathcal{A}[\![a_1]\!]s \leq \mathcal{A}[\![a_2]\!]s \\ \mathbf{ff} & \text{if } \mathcal{A}[\![a_1]\!]s > \mathcal{A}[\![a_2]\!]s \end{cases}$$

$$\mathcal{B}[\![\neg\ b]\!]s = \begin{cases} \mathbf{tt} & \text{if } \mathcal{B}[\![b]\!]s = \mathbf{ff} \\ \mathbf{ff} & \text{if } \mathcal{B}[\![b]\!]s = \mathbf{tt} \end{cases}$$

$$\mathcal{B}[\![b_1 \wedge b_2]\!]s = \begin{cases} \mathbf{tt} & \text{if } \mathcal{B}[\![b_1]\!]s = \mathbf{tt} \text{ and } \mathcal{B}[\![b_2]\!]s = \mathbf{tt} \\ \mathbf{ff} & \text{if } \mathcal{B}[\![b_1]\!]s = \mathbf{ff} \text{ or } \mathcal{B}[\![b_2]\!]s = \mathbf{ff} \end{cases}$$

Free Variables

$$\begin{aligned} \mathrm{FV}(n) &= \emptyset \\ \mathrm{FV}(x) &= \{\, x \,\} \\ \mathrm{FV}(a_1 + a_2) &= \mathrm{FV}(a_1) \cup \mathrm{FV}(a_2) \\ \mathrm{FV}(a_1 \star a_2) &= \mathrm{FV}(a_1) \cup \mathrm{FV}(a_2) \\ \mathrm{FV}(a_1 - a_2) &= \mathrm{FV}(a_1) \cup \mathrm{FV}(a_2) \end{aligned}$$

$\mathrm{FV}(\texttt{x+1}) = \{\, \texttt{x} \,\}$ and $\mathrm{FV}(\texttt{x+y}\star\texttt{x}) = \{\, \texttt{x}, \texttt{y} \,\}$

Substitutions

$$\begin{aligned} n[y \mapsto a_0] &= n \\ x[y \mapsto a_0] &= \begin{cases} a_0 & \text{if } x = y \\ x & \text{if } x \neq y \end{cases} \\ (a_1 + a_2)[y \mapsto a_0] &= (a_1[y \mapsto a_0]) + (a_2[y \mapsto a_0]) \\ (a_1 \star a_2)[y \mapsto a_0] &= (a_1[y \mapsto a_0]) \star (a_2[y \mapsto a_0]) \\ (a_1 - a_2)[y \mapsto a_0] &= (a_1[y \mapsto a_0]) - (a_2[y \mapsto a_0]) \end{aligned}$$

$(\texttt{x+1})[\texttt{x} \mapsto \texttt{3}] = \texttt{3+1}$ and $(\texttt{x+y}\star\texttt{x})[\texttt{x} \mapsto \texttt{y}-\texttt{5}] = (\texttt{y}-\texttt{5})+\texttt{y}\star(\texttt{y}-\texttt{5})$

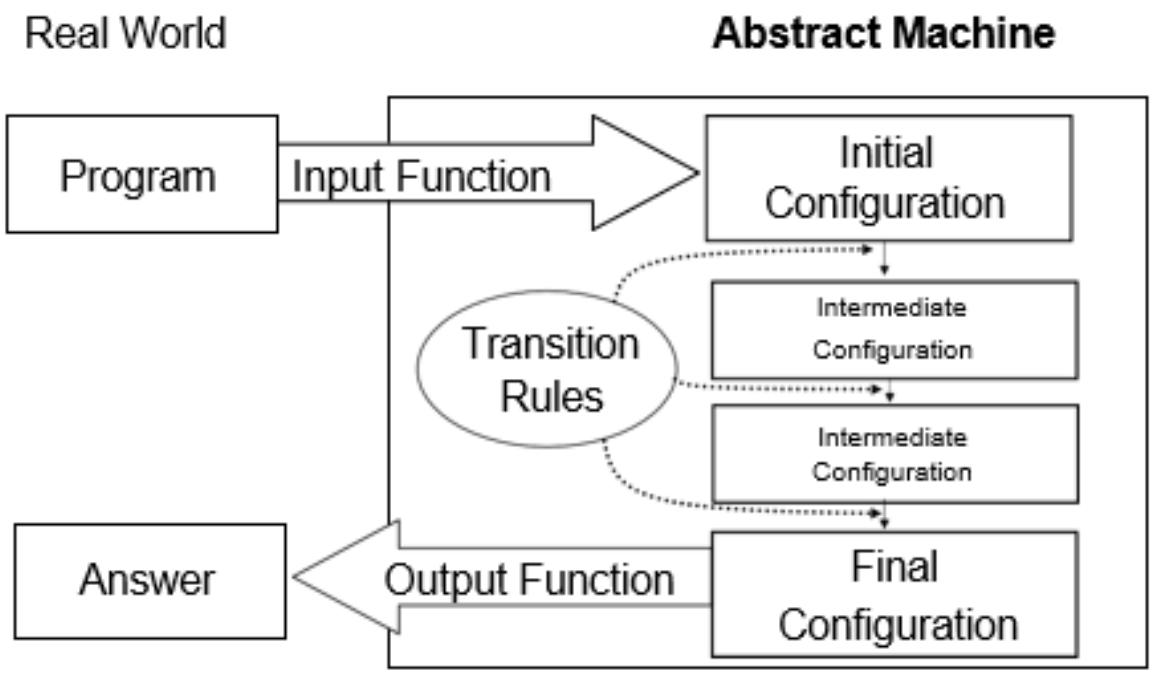
Operational Semantics Types

– Natural semantics: Its purpose is to describe how the overall results of executions are obtained; sometimes it is called a big-step operational semantics.

– Structural operational semantics: Its purpose is to describe how the individual steps of the computations take place; sometimes it is called a small-step operational semantics.

For both kinds of operational semantics, the meaning of statements will be specified by a transition system. It will have two types of configurations

Natural semantics for While



$[\text{ass}_{\text{ns}}]$ $\quad \langle x := a,\ s\rangle \to s[x \mapsto \mathcal{A}[\![a]\!]s]$

$[\text{skip}_{\text{ns}}]$ $\quad \langle \texttt{skip},\ s\rangle \to s$

$[\text{comp}_{\text{ns}}]$ $\quad \dfrac{\langle S_1,\ s\rangle \to s',\ \langle S_2,\ s'\rangle \to s''}{\langle S_1;S_2,\ s\rangle \to s''}$

$[\text{if}^{\text{tt}}_{\text{ns}}]$ $\quad \dfrac{\langle S_1,\ s\rangle \to s'}{\langle \texttt{if}\ b\ \texttt{then}\ S_1\ \texttt{else}\ S_2,\ s\rangle \to s'}$ if $\mathcal{B}[\![b]\!]s = \mathbf{tt}$

$[\text{if}^{\text{ff}}_{\text{ns}}]$ $\quad \dfrac{\langle S_2,\ s\rangle \to s'}{\langle \texttt{if}\ b\ \texttt{then}\ S_1\ \texttt{else}\ S_2,\ s\rangle \to s'}$ if $\mathcal{B}[\![b]\!]s = \mathbf{ff}$

$[\text{while}^{\text{tt}}_{\text{ns}}]$ $\quad \dfrac{\langle S,\ s\rangle \to s',\ \langle \texttt{while}\ b\ \texttt{do}\ S,\ s'\rangle \to s''}{\langle \texttt{while}\ b\ \texttt{do}\ S,\ s\rangle \to s''}$ if $\mathcal{B}[\![b]\!]s = \mathbf{tt}$

$[\text{while}^{\text{ff}}_{\text{ns}}]$ $\quad \langle \texttt{while}\ b\ \texttt{do}\ S,\ s\rangle \to s$ if $\mathcal{B}[\![b]\!]s = \mathbf{ff}$

$\langle S,\ s\rangle$ representing that the statement S is to be executed from the state s and

s representing a terminal (that is final) state.

Natural semantics

In a natural semantics we are concerned with the relationship between the initial and the final state of an execution.

Therefore the transition relation will specify the relationship between the initial state and the final state for each statement.

transition as <S, s → s'>

Intuitively this means that the execution of S from s will terminate and the resulting state will be s . The definition of → is given by the rules.

$$\frac{\langle S_1,\ s_1\rangle \rightarrow s_1',\ \cdots,\ \langle S_n,\ s_n\rangle \rightarrow s_n'}{\langle S,\ s\rangle \rightarrow s'} \quad \text{if } \cdots$$

where $S_1, \cdots, S_n$ are *immediate constituents* of S or are statements *constructed from* the immediate constituents of S. A rule has a number of *premises* (written above the solid line) and one *conclusion* (written below the solid line). A rule may also have a number of *conditions* (written to the right of the solid line) that have to be fulfilled whenever the rule is applied. Rules with an empty set of premises are called *axioms* and the solid line is then omitted.

Natural semantics

Examples

$$\langle \mathtt{x} := \mathtt{x}+1,\ s_0 \rangle \rightarrow s_0[\mathtt{x} \mapsto \mathbf{1}]$$

$$\langle \mathtt{skip},\ s_0 \rangle \rightarrow s_0$$

$$\frac{\langle \mathtt{skip},\ s_0 \rangle \rightarrow s_0,\ \langle \mathtt{x} := \mathtt{x}+1,\ s_0 \rangle \rightarrow s_0[\mathtt{x} \mapsto \mathbf{1}]}{\langle \mathtt{skip};\ \mathtt{x} := \mathtt{x}+1,\ s_0 \rangle \rightarrow s_0[\mathtt{x} \mapsto \mathbf{1}]} \longrightarrow \frac{\langle \mathtt{skip},\ s_0 \rangle \rightarrow s_0[\mathtt{x} \mapsto \mathbf{5}],\ \langle \mathtt{x} := \mathtt{x}+1,\ s_0[\mathtt{x} \mapsto \mathbf{5}] \rangle \rightarrow s_0}{\langle \mathtt{skip};\ \mathtt{x} := \mathtt{x}+1,\ s_0 \rangle \rightarrow s_0}$$

$$\frac{\langle \mathtt{skip},\ s_0 \rangle \rightarrow s_0}{\langle \mathtt{if}\ \mathtt{x} = 0\ \mathtt{then}\ \mathtt{skip}\ \mathtt{else}\ \mathtt{x} := \mathtt{x}+1,\ s_0 \rangle \rightarrow s_0}$$

When we use the axioms and rules to derive a transition <S, s → s'> , we obtain a derivation tree.
The root of the derivation tree is <S, s → s'> and the leaves are instances of axioms.
The internal nodes are conclusions of instantiated rules, and they have the corresponding premises as their immediate sons.

Example

(z:= x; x:= y); y:= z with state x=5, z=0 and y=7. The derivation tree is:

$$\dfrac{\dfrac{\langle \mathrm{z{:=}x},\ s_0\rangle \to s_1 \qquad \langle \mathrm{x{:=}y},\ s_1\rangle \to s_2}{\langle \mathrm{z{:=}x;\ x{:=}y},\ s_0\rangle \to s_2} \qquad \langle \mathrm{y{:=}z},\ s_2\rangle \to s_3}{\langle (\mathrm{z{:=}x;\ x{:=}y});\ \mathrm{y{:=}z},\ s_0\rangle \to s_3}$$

$$s_1 = s_0[\mathrm{z}\mapsto \mathbf{5}]$$
$$s_2 = s_1[\mathrm{x}\mapsto \mathbf{7}]$$
$$s_3 = s_2[\mathrm{y}\mapsto \mathbf{5}]$$

$$\dfrac{\langle \mathrm{z{:=}x},\ s_0\rangle \to s_1,\ \langle \mathrm{x{:=}y},\ s_1\rangle \to s_2}{\langle \mathrm{z{:=}x;\ x{:=}y},\ s_0\rangle \to s_2}$$

$$\dfrac{\langle \mathrm{z{:=}x;\ x{:=}y},\ s_0\rangle \to s_2,\ \langle \mathrm{y{:=}z},\ s_2\rangle \to s_3}{\langle (\mathrm{z{:=}x;\ x{:=}y});\ \mathrm{y{:=}z},\ s_0\rangle \to s_3}$$

combine the internal node <z:=x; x:=y, s0> → s2 and the leaf <y:=z, s2 >→ s3 with the root (z:=x; x:=y); y:=z, s0> → s3

$\texttt{y:=1; while } \neg(\texttt{x=1}) \texttt{ do } (\texttt{y:=y} \star \texttt{ x; x:=x}-1)$ state s x=3

$\langle \texttt{y:=1; while } \neg(\texttt{x=1}) \texttt{ do } (\texttt{y:=y} \star \texttt{ x; x:=x}-1), s\rangle \rightarrow s[\texttt{y}\mapsto\mathbf{6}][\texttt{x}\mapsto\mathbf{1}] \quad (*)$

Root node for the derivation tree

$\langle \texttt{y:=1; while } \neg(\texttt{x=1}) \texttt{ do } (\texttt{y:=y} \star \texttt{ x; x:=x}-1), s\rangle \rightarrow s_{61}$

$$\frac{\langle \texttt{y:=1}, s\rangle \rightarrow s_{13} \qquad T_1 \qquad \langle \texttt{while } \neg(\texttt{x=1}) \texttt{ do } (\texttt{y:=y}\star\texttt{x; x:=x}-1), s_{13}\rangle \rightarrow s_{61}}{\langle \texttt{y:=1; while } \neg(\texttt{x=1}) \texttt{ do } (\texttt{y:=y}\star\texttt{x; x:=x}-1), s\rangle \rightarrow s_{61}} \quad (**)$$

Since $\langle \texttt{y:=1}, s\rangle \rightarrow s_{13}$ has to be an instance of the axiom $[\text{ass}_{\text{ns}}]$, we get that $s_{13} = s[\texttt{y}\mapsto\mathbf{1}]$.

T_2 T_3

$$\langle \texttt{while}\ \neg(\texttt{x}{=}1)\ \texttt{do}\ (\texttt{y}{:=}\texttt{y}{\star}\texttt{x};\ \texttt{x}{:=}\texttt{x}{-}1),\ s_{13}\rangle \to s_{61}$$

root root

$$\langle \texttt{y}{:=}\texttt{y}{\star}\texttt{x};\ \texttt{x}{:=}\texttt{x}{-}1,\ s_{13}\rangle \to s_{32} \qquad \langle \texttt{while}\ \neg(\texttt{x}{=}1)\ \texttt{do}\ (\texttt{y}{:=}\texttt{y}{\star}\texttt{x};\ \texttt{x}{:=}\texttt{x}{-}1),\ s_{32}\rangle \to s_{61}$$

$$\frac{\langle \texttt{y}{:=}\texttt{y}{\star}\texttt{x},\ s_{13}\rangle \to s_{33} \qquad \langle \texttt{x}{:=}\texttt{x}{-}1,\ s_{33}\rangle \to s_{32}}{\langle \texttt{y}{:=}\texttt{y}{\star}\texttt{x};\ \texttt{x}{:=}\texttt{x}{-}1,\ s_{13}\rangle \to s_{32}}$$

$$\frac{\langle \texttt{y}{:=}\texttt{y}{\star}\texttt{x},\ s_{32}\rangle \to s_{62} \qquad \langle \texttt{x}{:=}\texttt{x}{-}1,\ s_{62}\rangle \to s_{61}}{\langle \texttt{y}{:=}\texttt{y}{\star}\texttt{x};\ \texttt{x}{:=}\texttt{x}{-}1,\ s_{32}\rangle \to s_{61}} \qquad \frac{\langle \texttt{y}{:=}\texttt{y}{\star}\texttt{x};\ \texttt{x}{:=}\texttt{x}{-}1,\ s_{32}\rangle \to s_{61} \qquad T_4}{\langle \texttt{while}\ \neg(\texttt{x}{=}1)\ \texttt{do}\ (\texttt{y}{:=}\texttt{y}{\star}\texttt{x};\ \texttt{x}{:=}\texttt{x}{-}1),\ s_{32}\rangle \to s_{61}}$$

where $s_{62} = s[\texttt{y}{\mapsto}\mathbf{6}][\texttt{x}{\mapsto}\mathbf{2}]$, $s_{61} = s[\texttt{y}{\mapsto}\mathbf{6}][\texttt{x}{\mapsto}\mathbf{1}]$, and T_4 is a derivation tree with root

$$\langle \texttt{while}\ \neg(\texttt{x}{=}1)\ \texttt{do}\ (\texttt{y}{:=}\texttt{y}{\star}\texttt{x};\ \texttt{x}{:=}\texttt{x}{-}1),\ s_{61}\rangle \to s_{61}$$

Function semantics

$$\mathcal{S}_{\text{ns}}\colon \mathbf{Stm} \to (\mathbf{State} \hookrightarrow \mathbf{State})$$

this means that for every statement S we have a partial function

$$\mathcal{S}_{\text{ns}}[\![S]\!] \in \mathbf{State} \hookrightarrow \mathbf{State}.$$

$$\mathcal{S}_{\text{ns}}[\![S]\!]s = \begin{cases} s' & \text{if } \langle S, s\rangle \to s' \\ \underline{\text{undef}} & \text{otherwise} \end{cases}$$

Induction

Induction on the Shape of Derivation Trees

1: Prove that the property holds for all the simple derivation trees by showing that it holds for the *axioms* of the transition system.

2: Prove that the property holds for all composite derivation trees: For each *rule* assume that the property holds for its premises (this is called the *induction hypothesis*) and prove that it also holds for the conclusion of the rule provided that the conditions of the rule are satisfied.

Structural semantics

The transition relation has a form

$< S, s > \Rightarrow \gamma$

γ either is of the form $\langle S', s' \rangle$ or of the form s'

Intermediate configuration (stuck/terminal configuration)

Final state

$[\text{ass}_{\text{sos}}] \quad \langle x := a, s \rangle \Rightarrow s[x \mapsto \mathcal{A}[\![a]\!]s]$

$[\text{skip}_{\text{sos}}] \quad \langle \texttt{skip}, s \rangle \Rightarrow s$

$$[\text{comp}^1_{\text{sos}}] \quad \frac{\langle S_1, s \rangle \Rightarrow \langle S_1', s' \rangle}{\langle S_1; S_2, s \rangle \Rightarrow \langle S_1'; S_2, s' \rangle}$$

$$[\text{comp}^2_{\text{sos}}] \quad \frac{\langle S_1, s \rangle \Rightarrow s'}{\langle S_1; S_2, s \rangle \Rightarrow \langle S_2, s' \rangle}$$

$[\text{if}^{\text{tt}}_{\text{sos}}] \quad \langle \texttt{if } b \texttt{ then } S_1 \texttt{ else } S_2, s \rangle \Rightarrow \langle S_1, s \rangle \text{ if } \mathcal{B}[\![b]\!]s = \mathbf{tt}$

$[\text{if}^{\text{ff}}_{\text{sos}}] \quad \langle \texttt{if } b \texttt{ then } S_1 \texttt{ else } S_2, s \rangle \Rightarrow \langle S_2, s \rangle \text{ if } \mathcal{B}[\![b]\!]s = \mathbf{ff}$

$[\text{while}_{\text{sos}}] \quad \langle \texttt{while } b \texttt{ do } S, s \rangle \Rightarrow$

$\langle \texttt{if } b \texttt{ then } (S; \texttt{while } b \texttt{ do } S) \texttt{ else skip}, s \rangle$

A *derivation sequence* of a statement S starting in state s is either

Finite sequence

$\gamma_0 \Rightarrow \gamma_1 \Rightarrow \gamma_2 \Rightarrow \cdots \Rightarrow \gamma_k$

Infinite sequence

$\gamma_0 \Rightarrow \gamma_1 \Rightarrow \gamma_2 \Rightarrow \cdots$

$$(\mathtt{z} := \mathtt{x};\ \mathtt{x} := \mathtt{y});\ \mathtt{y} := \mathtt{z}$$

Derivation sequence

$$\langle (\mathtt{z} := \mathtt{x};\ \bar{}\mathtt{x} := \mathtt{y});\ \mathtt{y} := \mathtt{z},\ s_0 \rangle$$

$$\Rightarrow \langle \mathtt{x} := \mathtt{y};\ \mathtt{y} := \mathtt{z},\ s_0[\mathtt{z} \mapsto \mathbf{5}] \rangle$$

$$\Rightarrow \langle \mathtt{y} := \mathtt{z},\ (s_0[\mathtt{z} \mapsto \mathbf{5}])[\mathtt{x} \mapsto \mathbf{7}] \rangle$$

$$\Rightarrow ((s_0[\mathtt{z} \mapsto \mathbf{5}])[\mathtt{x} \mapsto \mathbf{7}])[\mathtt{y} \mapsto \mathbf{5}]$$

Derivation tree

$$\langle (\mathtt{z} := \mathtt{x};\ \mathtt{x} := \mathtt{y});\ \mathtt{y} := \mathtt{z},\ s_0 \rangle \Rightarrow \langle \mathtt{x} := \mathtt{y};\ \mathtt{y} := \mathtt{z},\ s_0[\mathtt{z} \mapsto \mathbf{5}] \rangle$$

$$\frac{\dfrac{\langle \mathtt{z} := \mathtt{x},\ s_0 \rangle \Rightarrow s_0[\mathtt{z} \mapsto \mathbf{5}]}{\langle \mathtt{z} := \mathtt{x};\ \mathtt{x} := \mathtt{y},\ s_0 \rangle \Rightarrow \langle \mathtt{x} := \mathtt{y},\ s_0[\mathtt{z} \mapsto \mathbf{5}] \rangle}}{\langle (\mathtt{z} := \mathtt{x};\ \mathtt{x} := \mathtt{y});\ \mathtt{y} := \mathtt{z},\ s_0 \rangle \Rightarrow \langle \mathtt{x} := \mathtt{y};\ \mathtt{y} := \mathtt{z},\ s_0[\mathtt{z} \mapsto \mathbf{5}] \rangle}$$

$\langle \texttt{y:=1; while } \neg(\texttt{x=1}) \texttt{ do } (\texttt{y:=y} \star \texttt{x; x:=x-1}),\ s \rangle$ s x=3

configuration

$\langle \texttt{while } \neg(\texttt{x=1}) \texttt{ do } (\texttt{y:=y} \star \texttt{x; x:=x-1}),\ s[\texttt{y} \mapsto \mathbf{1}] \rangle$

This can be achieved by using assignment axiom and composition rule

Derivation tree

$$\frac{\langle \texttt{y:=1},\ s \rangle \Rightarrow s[\texttt{y} \mapsto \mathbf{1}]}{\langle \texttt{y:=1; while } \neg(\texttt{x=1}) \texttt{ do } (\texttt{y:=y} \star \texttt{x; x:=x-1}),\ s \rangle \Rightarrow \langle \texttt{while } \neg(\texttt{x=1}) \texttt{ do } (\texttt{y:=y} \star \texttt{x; x:=x-1}),\ s[\texttt{y} \mapsto \mathbf{1}] \rangle}$$

In the next step of the execution will rewrite the loop as a conditional using the axiom [while] to get the configuration

$$\langle \texttt{if } \neg(\texttt{x=1}) \texttt{ then } ((\texttt{y:=y} \star \texttt{x; x:=x-1}); \texttt{while } \neg(\texttt{x=1}) \texttt{ do } (\texttt{y:=y} \star \texttt{x; x:=x-1})) \texttt{ else skip},\ s[\texttt{y} \mapsto \mathbf{1}] \rangle$$

[if] yields new configuration

$\langle (\mathtt{y}{:=}\mathtt{y}{\star}\mathtt{x};\ \mathtt{x}{:=}\mathtt{x}{-}1);\ \mathtt{while}\ \neg(\mathtt{x}{=}1)\ \mathtt{do}\ (\mathtt{y}{:=}\mathtt{y}\star\mathtt{x};\ \mathtt{x}{:=}\mathtt{x}{-}1),\ s[\mathtt{y}{\mapsto}\mathbf{1}]\rangle$ use [ass], [comp 2] to obtain next configuration

$\langle \mathtt{while}\ \neg(\mathtt{x}{=}1)\ \mathtt{do}\ (\mathtt{y}{:=}\mathtt{y}\star\mathtt{x};\ \mathtt{x}{:=}\mathtt{x}{-}1),\ s[\mathtt{y}{\mapsto}\mathbf{3}][\mathtt{x}{\mapsto}\mathbf{2}]\rangle$

use [ass], [comp 2], and [comp 1] to obtain the configuration

$\langle \mathtt{x}{:=}\mathtt{x}{-}1;\ \mathtt{while}\ \neg(\mathtt{x}{=}1)\ \mathtt{do}\ (\mathtt{y}{:=}\mathtt{y}\star\mathtt{x};\ \mathtt{x}{:=}\mathtt{x}{-}1),\ s[\mathtt{y}{\mapsto}\mathbf{3}]\rangle$

Derivation tree

eventually reach the final state $s[\mathtt{y}{\mapsto}\mathbf{6}][\mathtt{x}{\mapsto}\mathbf{1}]$

$$\dfrac{\dfrac{\langle \mathtt{y}{:=}\mathtt{y}\star\mathtt{x},\ s[\mathtt{y}{\mapsto}\mathbf{1}]\rangle \Rightarrow s[\mathtt{y}{\mapsto}\mathbf{3}]}{\langle \mathtt{y}{:=}\mathtt{y}\star\mathtt{x};\ \mathtt{x}{:=}\mathtt{x}{-}1,\ s[\mathtt{y}{\mapsto}\mathbf{1}]\rangle \Rightarrow \langle \mathtt{x}{:=}\mathtt{x}{-}1,\ s[\mathtt{y}{\mapsto}\mathbf{3}]\rangle}}{\begin{array}{c}\langle (\mathtt{y}{:=}\mathtt{y}\star\mathtt{x};\ \mathtt{x}{:=}\mathtt{x}{-}1);\ \mathtt{while}\ \neg(\mathtt{x}{=}1)\ \mathtt{do}\ (\mathtt{y}{:=}\mathtt{y}\star\mathtt{x};\ \mathtt{x}{:=}\mathtt{x}{-}1),\ s[\mathtt{y}{\mapsto}\mathbf{1}]\rangle \Rightarrow \\ \langle \mathtt{x}{:=}\mathtt{x}{-}1;\ \mathtt{while}\ \neg(\mathtt{x}{=}1)\ \mathtt{do}\ (\mathtt{y}{:=}\mathtt{y}\star\mathtt{x};\ \mathtt{x}{:=}\mathtt{x}{-}1),\ s[\mathtt{y}{\mapsto}\mathbf{3}]\rangle\end{array}}$$

Induction

Induction on the Length of Derivation Sequences

1: Prove that the property holds for all derivation sequences of length 0.

2: Prove that the property holds for all finite derivation sequences: Assume that the property holds for all derivation sequences of length at most k (this is called the *induction hypothesis*) and show that it holds for derivation sequences of length k+1.